RECURSO DIDÁCTICO DE ALTO RENDIMIENTO · JAVASCRIPT CLIENTE

⏱️ TEMPORIZADORES EN JAVASCRIPT

Manual de referencia rápida y arquitectura asíncrona: domina el control del tiempo en el navegador mediante `setTimeout()`, `clearTimeout()`, `setInterval()` y `clearInterval()` con analogías de aula, cajitas de memoria y patrones profesionales.

⏳ setTimeout()

Ejecución diferida de una sola vez tras un plazo de milisegundos. Ideal para avisos temporales y retrasos calculados.

🛑 clearTimeout()

Cancelación segura de cuentas atrás pendientes antes de su vencimiento utilizando su ticket identificador.

🔁 setInterval()

Bucle periódico repetitivo a intervalos regulares. Base para cronómetros, relojes y animaciones frame a frame.

⏹️ clearInterval()

Detención definitiva del ciclo repetitivo evitando consumos de memoria y bucles infinitos en el navegador.

👨‍🏫 **Ramón Abramo**
Profesor especialista con más de 25 años de experiencia docente

Módulo Inicial · Guía Técnica
Santander / Confección Web 2026

📑 Índice General de Contenidos (Haz clic para saltar)

Manual de Referencia Rápida

1. ¿Cómo controla el tiempo el navegador? Pág 2
2. Las 4 Funciones Fundamentales Pág 2
3. Pizarra de Memoria y Tickets ID Pág 3
4. Anatomía de setTimeout y clearTimeout Pág 3
5. Anatomía de setInterval y clearInterval Pág 3
6. Práctica 1: Aviso Retardado con Cancelación Pág 4
7. Práctica 2: Caja Creciente (Animación DOM) Pág 5
8. Práctica 3: Cronómetro Interactivo 3 Estados Pág 6
9. Práctica 4: Cuenta Atrás Coordinada Pág 7
10. Top 5 Errores Frecuentes y Soluciones Pág 7

⏱️ 1. ¿Cómo controla el tiempo el navegador?

Por defecto, JavaScript es un lenguaje síncrono que ejecuta las instrucciones una detrás de otra de manera inmediata en un único hilo. Sin embargo, en el desarrollo web moderno necesitamos programar eventos que ocurran en el futuro o se repitan periódicamente: avisos emergentes, cronómetros o animaciones. Para lograrlo, el navegador nos ofrece cuatro funciones asíncronas nativas mediante las Web APIs.

📏 **LA REGLA DE ORO DE LOS MILISEGUNDOS (MS)**

Todas las funciones de temporización en JavaScript miden el tiempo en **milisegundos** (`ms`). Un segundo equivale exactamente a **1.000 milisegundos** (`1s = 1000ms`, `2s = 2000ms`, `500ms = 0.5s`). ¡Escribir `5` significa 5 milisegundos (casi instantáneo), nunca 5 segundos!

📊 2. Las 4 Funciones Fundamentales: Tabla Comparativa

FUNCIÓN	COMPORTAMIENTO	¿QUÉ DEVUELVE?	¿CÓMO SE DETIENE?
SETTIMEOUT(FN, MS)	Espera `ms` y ejecuta la función **una sola vez**.	Un número entero (**ID** identificador).	Con `clearTimeout(id)` antes de cumplirse el plazo.
CLEARTIMEOUT(ID)	Cancela la alarma diferida de un `setTimeout`.	Nada (`undefined`).	No aplica (es la función destructora).
SETINTERVAL(FN, MS)	Ejecuta la función **repetidamente** cada `ms`.	Un número entero (**ID** de intervalo).	Con `clearInterval(id)` en cualquier momento.
CLEARINTERVAL(ID)	Frena el bucle repetitivo de un `setInterval`.	Nada (`undefined`).	No aplica (es la función de parada).

🧠 3. La Pizarra de Memoria: ¿Qué es el Identificador (ID)?

Cuando solicitas al navegador un temporizador, este no te devuelve la función ni el tiempo; te entrega un **número de ticket único (ID)** (por ejemplo: `1`, `2`, `3`...). Si no guardas ese ticket en una variable de control, perderás la referencia y ya no podrás cancelarlo.

VARIABLE EN MEMORIA	TICKET DEL NAVEGADOR	VARIABLE EN MEMORIA	TICKET DEL NAVEGADOR
`let idAlarma`	`ID: 1 (Timeout)`	`let idReloj`	`ID: 2 (Interval)`

⏳ 4. Anatomía de la Ejecución Única: setTimeout() y clearTimeout()

> 🍳 **Analogía de Pizarra: La Alarma del Horno**
> Programas el horno para dentro de 3 segundos: cuando el tiempo se cumple, suena un pitido y se apaga solo. Si decides cancelar antes de que suene porque vas a salir de casa, necesitas pulsar el botón de parada usando el identificador guardado.

```
// ⏰ PASO 1: Programamos la acción diferida y guardamos su ID en una variable
const idAlarma = setTimeout(() => {
    console.log("🔔 ¡Han pasado 3 segundos! Acción ejecutada una sola vez.");
}, 3000);

// 🛑 PASO 2 (Opcional): Si queremos abortarla antes de cumplirse los 3 segundos
clearTimeout(idAlarma);
console.log("🛑 Alarma abortada a tiempo. La función nunca llegará a ejecutarse.");
```

🔄 5. Anatomía de la Ejecución Periódica: setInterval() y clearInterval()

> 🎵 **Analogía de Pizarra: El Metrónomo Musical**
> Funciona exactamente igual que un segundero o un metrónomo: cada vez que transcurre el intervalo configurado vuelve a sonar de forma indefinida hasta que le apliquemos un freno explícito mediante `clearInterval()`.

```
// 🔄 PASO 1: Iniciamos la repetición periódica cada 1000 ms (1 segundo)
let vueltas = 0;
const idReloj = setInterval(() => {
    vueltas++;
    console.log("⏰ Vuelta número: " + vueltas);

    // 🛑 PASO 2: Condición de parada para evitar un bucle infinito en memoria
    if (vueltas === 5) {
        clearInterval(idReloj);
        console.log("🏁 Meta alcanzada: Intervalo detenido automáticamente.");
    }
}, 1000);
```

6. Práctica de Referencia 1: Aviso Retardado con Cancelación

OBJETIVO Y FUNCIONES EMPLEADAS

Funciones: `setTimeout()` + `clearTimeout()` · **Reto:** Simular el sistema de envío de un correo que ofrece una ventana de 4 segundos al usuario para pulsar «Cancelar / Deshacer envío» antes de que el mensaje parta definitivamente.

Estructura HTML y Preparación

```
<!-- 🧱 Estructura HTML de la Práctica 1 -->
<button id="btnEnviar">📩 Enviar mensaje (4s)</button>
<button id="btnCancelar" disabled>🔴 Cancelar envío</button>
<p id="estado">Esperando acción...</p>
```

Código JavaScript Comentado Paso a Paso

```
// 🎯 PASO 1: Capturamos los elementos del DOM
const btnEnviar = document.getElementById("btnEnviar");
const btnCancelar = document.getElementById("btnCancelar");
const estado = document.getElementById("estado");

// 🧠 PASO 2: Variable global en el ámbito superior para compartir el ID
let temporizadorEnvio = null;

// 🚀 PASO 3: Escuchador para iniciar el envío diferido
btnEnviar.addEventListener("click", () => {
    estado.textContent = "⏳ Enviando en 4 segundos... Puedes cancelar.";
    btnEnviar.disabled = true;
    btnCancelar.disabled = false;

    // Programamos la acción diferida tras 4000 ms
    temporizadorEnvio = setTimeout(() => {
        estado.textContent = "✅ ¡Mensaje enviado con éxito al servidor!";
        btnCancelar.disabled = true;
        btnEnviar.disabled = false;
        temporizadorEnvio = null;
    }, 4000);
});

// 🛑 PASO 4: Escuchador para cancelar el envío antes de que se cumpla el tiempo
btnCancelar.addEventListener("click", () => {
    // Detenemos el setTimeout antes de que se cumpla el tiempo
    clearTimeout(temporizadorEnvio);
    temporizadorEnvio = null;

    estado.textContent = "🔴 Envío cancelado por el usuario. Permanece en borradores.";
    btnCancelar.disabled = true;
    btnEnviar.disabled = false;
});
```

Buena Práctica de Interfaz (UX en el Aula)

Alternar la propiedad `disabled` de los botones impide que el alumno haga doble clic y garantiza que solo se pueda cancelar cuando realmente haya una cuenta atrás activa en marcha.

📦 7. Práctica de Referencia 2: Animación de Caja Creciente

🎯 OBJETIVO Y FUNCIONES EMPLEADAS

Funciones: `setInterval()` + `clearInterval()` · **Reto:** Animar progresivamente el ancho de un elemento HTML sumando 5 píxeles cada 50 milisegundos y frenar automáticamente al alcanzar un límite de 300px.

🔍 Para Entenderlo: La Técnica Tradicional de Animación

Antes del estándar CSS3 (`transitions` y `@keyframes`), todo movimiento dinámico en pantalla se lograba modificando propiedades del DOM dentro de un `setInterval` a alta velocidad (20 a 60 veces por segundo).

Estructura HTML y CSS

```
<!-- 🧱 Elementos HTML con estilos en línea -->
<button id="btnCrecer">🚀 Hacer crecer la caja</button>
<div id="caja" style="width:50px; height:36px; background:#0284c7; color:#fff;
     text-align:center; line-height:36px; font-weight:bold; margin-top:8px;">50px</div>
```

Código JavaScript de la Animación

```
const btnCrecer = document.getElementById("btnCrecer");
const caja = document.getElementById("caja");

btnCrecer.addEventListener("click", () => {
    btnCrecer.disabled = true;
    let anchoActual = 50;

    // 💣 Cada 50 ms aumentamos el ancho en 5 píxeles (20 fotogramas por segundo)
    const idAnimacion = setInterval(() => {
        anchoActual += 5;

        // 🎨 Modificamos directamente el estilo en línea del DOM
        caja.style.width = anchoActual + "px";
        caja.textContent = anchoActual + "px";

        // 🛑 Freno automático: cuando alcanza o supera los 300px
        if (anchoActual >= 300) {
            clearInterval(idAnimacion);
            caja.style.background = "#047857";
            caja.textContent = "¡Completado (300px)!";
        }
    }, 50);
});
```

⚠️ Regla de Parada Segura (>= en vez de ===)

Utiliza siempre el operador `>=` en lugar de `===` para condiciones de parada de intervalos numéricos. Si por algún incremento el valor saltase a 302px, un `===` jamás se cumpliría y el bucle continuaría indefinidamente.

⏱️ 8. Práctica de Referencia 3: Cronómetro Interactivo de Tres Estados

🎯 OBJETIVO Y FUNCIONES EMPLEADAS

Funciones: `setInterval()` + `clearInterval()` · **Reto:** Controlar el ciclo de vida de un cronómetro mediante tres botones: *Arrancar*, *Pausar* y *Reiniciar*, blindando el código contra clics duplicados.

🔴 La Trampa Más Frecuente en Exámenes de Programación

Si el usuario pulsa *Arrancar* tres veces seguidas, se crearán tres intervalos paralelos. El contador aumentará a triple velocidad y la variable `idCronometro` solo recordará el último ID, dejando los dos anteriores huérfanos e imparables. ¡Para evitarlo, comprueba siempre `if (idCronometro !== null) return;`!

Estructura HTML y Código JavaScript Completo

```html
<!-- 🧱 Botonera de control del cronómetro -->
<h3>Segundos: <span id="pantallaContador">0</span>s</h3>
<button id="btnArrancar">▶ Arrancar</button>
<button id="btnParar">⏸ Parar</button>
<button id="btnReiniciar">🔄 Reiniciar</button>
```

```javascript
const pantalla = document.getElementById("pantallaContador");
const btnArrancar = document.getElementById("btnArrancar");
const btnParar = document.getElementById("btnParar");
const btnReiniciar = document.getElementById("btnReiniciar");

let segundos = 0;
let idCronometro = null; // null indica que el reloj está detenido

btnArrancar.addEventListener("click", () => {
    // 🛡️ SEGURIDAD ANTI-DUPLICADOS: Salimos si ya hay un reloj en marcha
    if (idCronometro !== null) return;

    idCronometro = setInterval(() => {
        segundos++;
        pantalla.textContent = segundos;
    }, 1000);
});

btnParar.addEventListener("click", () => {
    clearInterval(idCronometro);
    idCronometro = null; // 🔓 Crucial: Liberamos la variable para permitir reanudar
});

btnReiniciar.addEventListener("click", () => {
    clearInterval(idCronometro);
    idCronometro = null;
    segundos = 0;
    pantalla.textContent = "0";
});
```

🚀 9. Práctica de Referencia 4: Cuenta Atrás Coordinada

En aplicaciones profesionales es habitual coordinar ambos temporizadores: un `setInterval` descuenta los segundos y, cuando llega a cero, un `setTimeout` programa el reseteo automático diferido:

```
const btnCuenta = document.getElementById("btnCuentaAtras");
const visor = document.getElementById("visorCuenta");

btnCuenta.addEventListener("click", () => {
    btnCuenta.disabled = true;
    let restante = 5;
    visor.textContent = "⏳ Tiempo restante: " + restante + "s";

    const idCuenta = setInterval(() => {
        restante--;
        if (restante > 0) {
            visor.textContent = "⏳ Tiempo restante: " + restante + "s";
        } else {
            // 🛑 1. Frenamos el bucle repetitivo
            clearInterval(idCuenta);
            visor.textContent = "🚨 ¡TIEMPO AGOTADO!";

            // ⏳ 2. Programamos el reseteo único diferido a los 2 segundos
            setTimeout(() => {
                visor.textContent = "Listo para comenzar";
                btnCuenta.disabled = false;
            }, 2000);
        }
    }, 1000);
});
```

🛑 10. Top 5 Errores Frecuentes y Soluciones de Aula

ERROR HABITUAL	¿POR QUÉ OCURRE?	SOLUCIÓN RECOMENDADA
Ejecutar la función al pasarla `setTimeout(saludar(), 1000)`	Al escribir los paréntesis `()`, la función se ejecuta de inmediato sin esperar el tiempo.	Pasar solo el nombre de la función `setTimeout(saludar, 1000)` o una flecha `() ⇒ saludar()`.
Intervalos acelerados (Pulsar varias veces Arrancar)	Cada clic añade un nuevo `setInterval` concurrente; todos suman segundos a la vez.	Guardar el ID en una variable y validar `if (id ≠= null) return;` antes de iniciar otro.
Olvidar guardar el ID `setInterval(fn, 1000);`	Si no asignas el resultado numérico, pierdes la referencia y no podrás frenarlo.	Asignar siempre a variable: `const id = setInterval( ... )` o `idReloj = ...`.
Confundir segundos con ms `setTimeout(fn, 5)`	El parámetro espera milisegundos. `5` representa 5 ms (casi instantáneo), no 5 segundos.	Multiplicar siempre los segundos por 1.000: `5 * 1000 = 5000`.
No poner id = null tras clearInterval	`clearInterval(id)` detiene el reloj pero no limpia el valor numérico de la variable.	Añadir siempre `id = null;` justo después de `clearInterval(id);`.